САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №2

«Высокопроизводительные вычисления в механике»

Выполнил: студент группы 4446-010303D

Погудин М.Э.

Проверил: Неженский М.С.

«____» _____2026 г.

Оценка _____

Самара 2026

Для выполнения работы, в коде необходимо раскоментировать строку:

omp_set_num_threads(2);

Также необходимо поменять #pragma for на #pragma omp for

Основная разница первой и второй лабораторной работы, в использовании потоков. Ниже представлена часть кода, добавленная в этой лабораторной работе.

```
int threads;
omp_set_num_threads(2);
#pragma omp parallel shared(threads)
{
threads = omp_get_num_threads();

#pragma omp for
for (int i=0; i < N; i++)
      for (int j = 0; j < N; j++)
   {
    C[i*N + j] = 0;
    for (int k = 0; k < N; k++)
    {
            C[i*N + j] += A[i*N + k] * B[k*N + j];
    }
   }
}
```

Произведем расчёты с помощью измененной версии кода и получим время расчета для матрицы размера 1000. Компиляция программы выполняется с помощью следующей команды:

g++ -fopenmp matMulOMP.cpp -o matMulOMP

Ниже приведен вывод результатов в терминале (см. Рисунок 1).

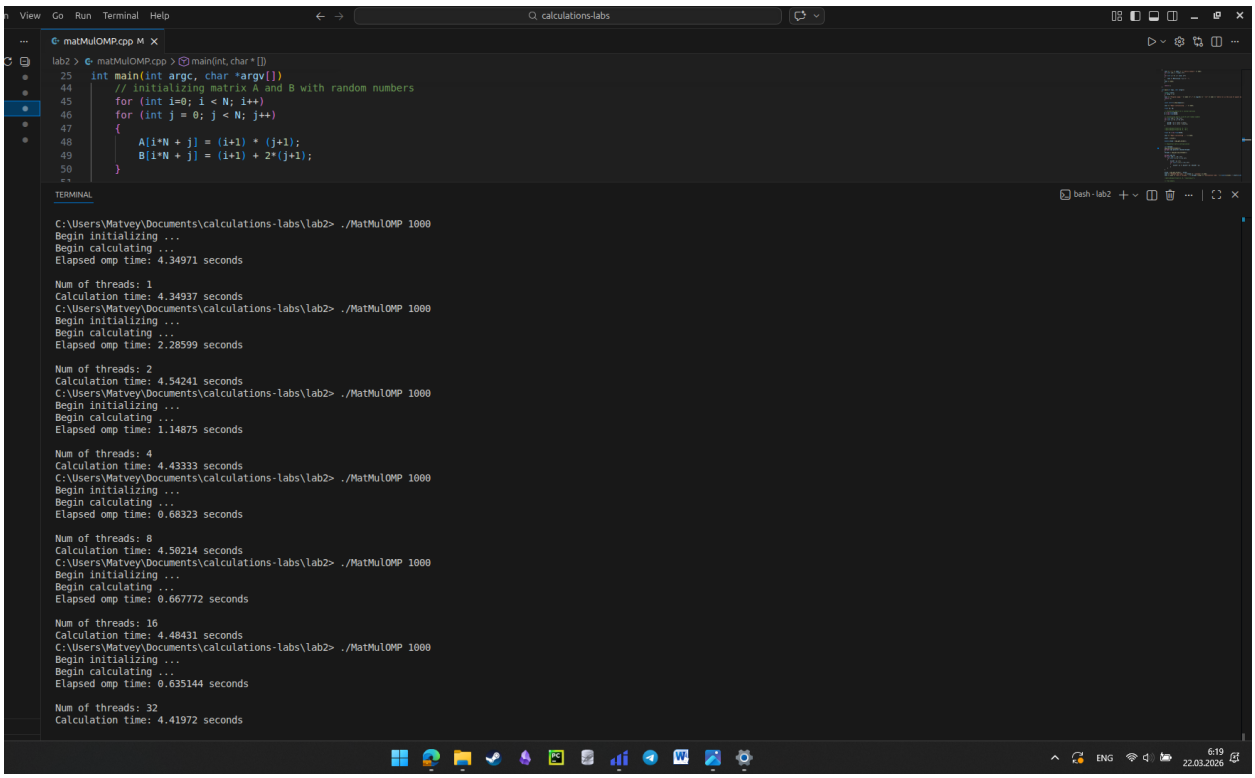


Рисунок 1 – Результаты расчета

По полученным результатам построим сравнительную таблицу (см. Таблица 1) и график (См. Рисунок 2).

Таблица 1 – Результаты вычислений для матрицы с n=1000

| Количество потоков | Calculation time, с | Elapsed omp time, с |
|---|---|---|
| 1 | 4,34937 | 4,34971 |
| 2 | 4,54241 | 2,28599 |
| 4 | 4,43333 | 1,14875 |
| 8 | 4,50241 | 0,68323 |
| 16 | 4,48431 | 0,667772 |
| 32 | 4,41972 | 0,635144 |

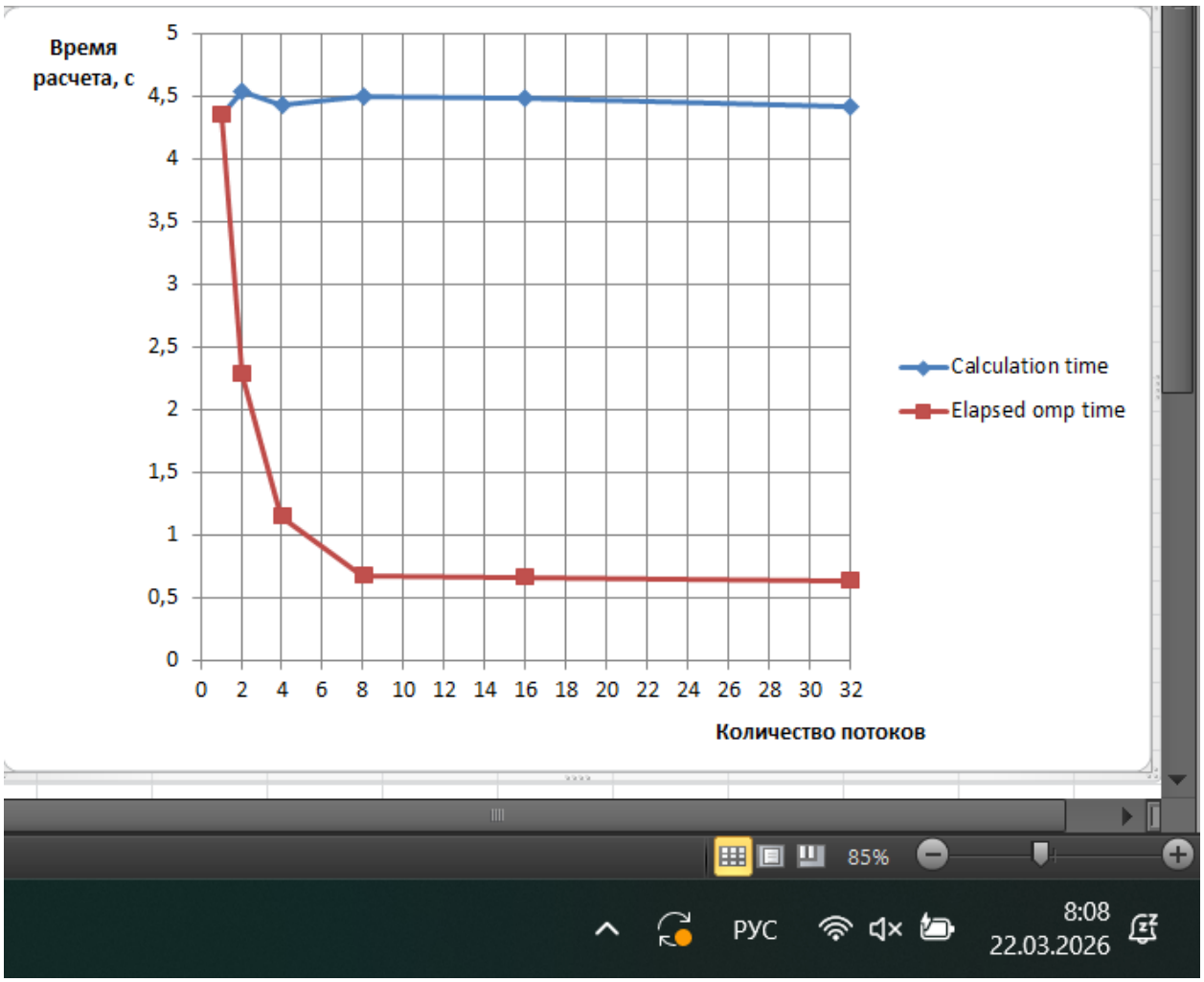


Рисунок 2 – График времени расчета

Elapsed omp time показывает реальное время работы программы с использованием OpenMP, тогда как Calculation time отражает суммарное время вычислений всех потоков. Как видно из рисунка 2, увеличение числа потоков выше 8 не даёт значительного ускорения из-за ограничения числа физических ядер.

Сравним результаты работы с результатами первой лабораторной работы при использовании различного количества потоков (см. Рисунок 3).

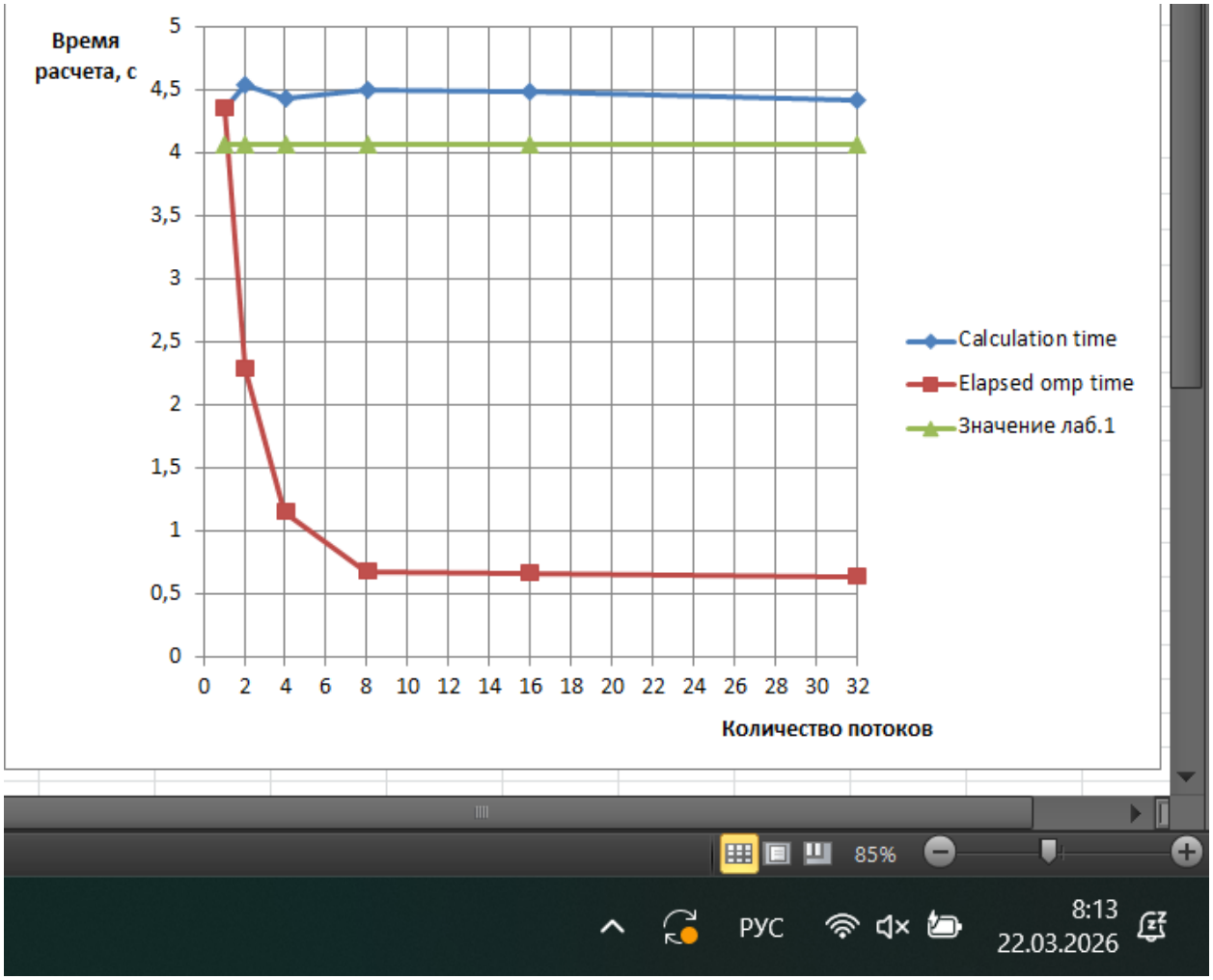


Рисунок 3 – Сравнительный график результатов вычислений при разном количестве потоков

## Вывод

Последовательное выполнение заняло 4,077 с и послужило эталоном для расчёта ускорения. При использовании одного потока OpenMP время составило 4,35 с, что обусловлено начальными накладными расходами библиотеки. С двумя потоками время сократилось до 2,29 с (почти вдвое), демонстрируя эффективность первичной параллелизации.

При четырёх потоках время уменьшилось до 1,15 с, при восьми — до 0,68 с, с близким к линейному ускорением, характерным для хорошо распараллеливаемых задач. Начиная с 16 потоков эффект замедляется: 0,67 с, а при 32 потоках — 0,64 с, что указывает на ограниченность преимуществ параллелизации.

Приложение

Ссылка на репозиторий с выполненными лабораторными работами: https://github.com/MatveyPogudin/high-performance-computing